

Списки контроля доступа

От битов rwx в Linux до IAM в AWS и GCP:
что должен понимать разработчик

Справочник к лабораторной работе №1 по дисциплине
«Проектирование защищённых интеллектуальных информационных
систем»

Содержание

1. Модель доступа: словарь, которым нужно владеть	4
1.1. Матрица доступа Лэмпсона	4
1.2. Два среза матрицы: ACL и capability	4
1.3. Семейство моделей: DAC, MAC, RBAC, ABAC, ReBAC	5
2. Права POSIX: то, что лежит в основе	6
2.1. Классы взаимно исключают друг друга	6
2.2. Биты на каталоге означают совсем не то, что на файле	6
2.3. Удаление файла не зависит от прав на файл	6
2.4. setuid, setgid и umask	7
2.5. Права суперпользователя не безграничны	7
2.6. Запуск скрипта и запуск бинарника — разные требования	7
2.7. Грабли, на которые наступают все	8
3. Настоящие POSIX ACL: setfacl и getfacl	9
3.1. Порядок разбора записей	9
3.2. Маска — главная ловушка POSIX ACL	9
3.3. Наследование: default ACL	10
4. Другая ветвь: ACL в NFSv4 и NTFS	11
5. Что стоит рядом с ACL в Linux	12
5.1. Linux capabilities: конец монолитного root	12
5.2. Мандатный слой: SELinux и AppArmor	12
5.3. seccomp: ограничение по системным вызовам	13
6. Разграничение доступа в СУБД	14
6.1. Роль — это и пользователь, и группа	14
6.2. Уровни, на которых выдаются права	14
6.3. PUBLIC и привилегии по умолчанию	14
6.4. Построчная защита	15
6.5. Грабли PostgreSQL	15
7. Авторизация в приложении	17
7.1. Три поколения моделей	17
7.2. Zanzibar: как это устроено в больших системах	17

7.3. Самая частая уязвимость: BOLA	17
8. AWS: политики, ACL и порядок их вычисления	19
8.1. Источники решения	19
8.2. Порядок вычисления — то, что нужно помнить наизусть	19
8.3. S3: почему ACL объявлены устаревшими	19
8.4. Security Group против Network ACL	20
8.5. Пути повышения привилегий и типовые ошибки	20
9. GCP: иерархия и объединение политик	22
9.1. Иерархия	22
9.2. Чем ограничивать, если объединение нельзя отменить	22
9.3. Роли и разрешения	22
9.4. Сервисные учётные записи — главный вектор эскалации	22
9.5. Cloud Storage: uniform против fine-grained	23
9.6. Сводное сравнение подходов	23
10. Kubernetes: RBAC и сетевые политики	24
10.1. RBAC умеет только разрешать	24
10.2. Пути повышения привилегий, которые нужно знать	24
10.3. NetworkPolicy: семантика по умолчанию	24
11. Сквозные принципы	25
12. Шпаргалка команд	26
Linux: базовые права	26
Linux: расширенные ACL и полномочия	26
PostgreSQL	26
Облака и Kubernetes	26
13. Что почитать дальше	27

1. Модель доступа: словарь, которым нужно владеть

Любая система разграничения доступа отвечает на один вопрос: можно ли **субъекту** выполнить **операцию** над **объектом**. Субъект — тот, кто действует: пользователь, процесс, сервис, микросервис с токеном. Объект — то, над чем действуют: файл, каталог, процесс, строка таблицы, бакет, виртуальная машина. Операция — чтение, запись, выполнение, удаление, изменение прав.

Различайте два понятия, которые постоянно путают. **Аутентификация** отвечает на вопрос «кто ты» и заканчивается установленной личностью. **Авторизация** отвечает на вопрос «что тебе можно» и заканчивается решением разрешить или отказать. Валидный токен не означает права на конкретный ресурс — это две независимые проверки, и вторую забывают гораздо чаще первой.

1.1. Матрица доступа Лэмпсона

Формально права описываются матрицей: строки — субъекты, столбцы — объекты, в ячейке — набор разрешённых операций. Матрица введена Батлером Лэмпсоном в 1971 году и остаётся базовой моделью до сих пор.

	file_a	file_b	process_x
alice	rw	r	kill
bob	r	—	—
service	r	rw	—

Хранить матрицу целиком невозможно: она разреженная и огромная. Поэтому её режут на срезы — и способ нарезки определяет всю архитектуру системы безопасности.

1.2. Два среза матрицы: ACL и capability

Список контроля доступа (ACL) — это столбец матрицы. Он хранится вместе с объектом и перечисляет, кому что можно с этим объектом делать. Каждая запись такого списка называется **ACE** (Access Control Entry). Права на файл в Linux, bucket policy в S3, IAM-политика на ресурсе в GCP — всё это ACL.

Список возможностей (capability list) — это строка матрицы. Он привязан к субъекту и представляет собой набор неподделяваемых «ключей»: владеешь ключом — можешь действовать, и системе не нужно выяснять, кто ты. Файловые дескрипторы, OAuth-токены, предподписанные ссылки S3, signed URL в GCS, JWT, токены сервисных учётных записей Kubernetes — всё это capability.

Свойство	ACL (столбец)	Capability (строка)
Где хранится	у объекта	у субъекта
Нужна ли личность субъекта	да, проверка при каждом доступе	нет, достаточно предъявить ключ
«Кто имеет доступ к X?»	тривиально: прочитать список	трудно: нужен обратный индекс
«К чему имеет доступ S?»	трудно: обойти все объекты	тривиально: посмотреть ключи
Делегирование	требует изменения объекта	передал ключ — и всё, часто бесконтрольно

Свойство	ACL (столбец)	Sarability (строка)
Отзыв прав	просто: убрал запись	сложно: ключ уже у получателя, нужен срок жизни или чёрный список

Практический вывод

Как только вы выдаёте предподписанную ссылку или долгоживущий токен, вы переходите из мира ACL в мир sarability и теряете дешёвый отзыв прав. Отсюда правило: срок жизни таких ключей должен быть минимально необходимым, а не «на всякий случай год».

1.3. Семейство моделей: DAC, MAC, RBAC, ABAC, ReBAC

Модель	Кто решает	Как выглядит	Где встречается
DAC — дискреционная	владелец объекта	владелец сам раздаёт права	права rwx в Linux, ACL в NTFS, GRANT в СУБД
MAC — мандатная	администратор политики, владелец не может её ослабить	метки и правила поверх DAC	SELinux, AppArmor, военные многоуровневые системы
RBAC — ролевая	администратор через роли	субъект → роль → набор прав	IAM-поли, роли PostgreSQL, Kubernetes RBAC
ABAC — атрибутная	политика, вычисляемая из атрибутов	правило над атрибутами субъекта, объекта, среды	IAM Conditions в GCP, Condition в политиках AWS, OPA
ReBAC — на отношениях	граф связей между объектами	«кто связан с объектом отношением X»	Google Zanzibar, SpiceDB, OpenFGA

Модели не исключают друг друга и почти всегда работают слоями. Файл в Linux под SELinux проверяется дважды: сначала DAC (биты и ACL), потом MAC (метки). Запрос в облаке проверяется четырежды: организационная политика, роль, условие политики и политика на самом ресурсе. **Отказ на любом слое означает отказ целиком** — это ключевое свойство, из которого следует, что отладка прав всегда начинается с вопроса «какой именно слой отказал».

2. Права POSIX: то, что лежит в основе

Классическая модель Unix хранит у каждого объекта файловой системы идентификатор владельца, идентификатор группы-владельца и девять битов прав — по три (r, w, x) для трёх классов: владельца, группы и остальных. Это самый маленький возможный ACL: ровно три записи.

2.1. Классы взаимно исключают друг друга

Ядро определяет класс субъекта по первому совпадению и дальше **не смотрит**: если вы владелец файла, применяются только биты владельца, даже если вы состоите в группе-владельце и у группы прав больше.

```
$ ls -l secret.txt
----rwx--- 1 alice developers 30 Sep  6 12:00 secret.txt

# alice — владелец, её класс "owner" с правами ---
# alice состоит в группе developers, но это уже не важно
$ sudo -u alice cat secret.txt
cat: secret.txt: Permission denied

# bob тоже в developers, но он не владелец -> класс "group" с правами rwx
$ sudo -u bob cat secret.txt
секретный текст
```

Проверьте себя

Файл с режимом 0070, принадлежащий alice:developers. Кто из них может его прочитать — alice, состоящая в developers, или bob, состоящий в developers? Ответ: только bob. Владелец попадает в класс owner с нулевыми правами и теряет доступ к собственному файлу.

2.2. Биты на каталоге означают совсем не то, что на файле

Бит	На файле	На каталоге
r	читать содержимое	получить список имён (ls), но не метаданные записей
w	изменять содержимое	создавать, удалять и переименовывать записи внутри
x	выполнять	проходить сквозь каталог: обращаться к записям по имени, заходить внутрь (cd), разрешать путь

Отсюда следуют два неочевидных, но постоянно используемых на практике факта.

- Каталог с правами --x (только выполнение) позволяет открыть /dir/known_name, если имя известно заранее, но не позволяет узнать, что в каталоге лежит. Так устроены домашние каталоги на shared-хостингах и каталоги вида /var/lib/..., куда нужно пускать по точному пути.
- Чтобы прочитать /a/b/c/file, нужен бит x на /a, /a/b и /a/b/c. Права на сам файл проверяются последними, и «Permission denied» чаще всего означает проблему не с файлом, а с каталогом выше по пути.

2.3. Удаление файла не зависит от прав на файл

Удаление — это изменение каталога, а не файла: из каталога убирается запись. Поэтому unlink требует w и x на **родительском каталоге**, а права самого файла не проверяются вовсе. Файл с режимом 000, принадлежащий root, спокойно удаляется любым пользователем, у которого есть запись в каталоге.

Именно поэтому существует **sticky bit** (t, восьмеричное 1000). На каталоге он ограничивает удаление и переименование: убрать запись может только владелец файла, владелец каталога или root. Классический пример — /tmp с режимом 1777: писать может каждый, а удалять чужое — нет.

```
$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Sep  6 12:00 /tmp
#          ^ sticky bit: общий каталог для записи, но без удаления чужого
```

2.4. setuid, setgid и umask

- **setuid** (4000) на исполняемом файле: процесс получает эффективный UID владельца файла. Так работает passwd, которому нужен доступ к /etc/shadow. Каждый setuid-бинарник — потенциальная точка повышения привилегий, поэтому их количество в системе стоит держать под контролем: `find / -perm -4000 -type f`.
- **Linux игнорирует setuid на скриптах.** Это сознательное решение: гонка между проверкой прав и открытием файла интерпретатором делала такие скрипты неустраимо уязвимыми. Работает только на бинарных файлах.
- **setgid** (2000) на каталоге: новые файлы внутри наследуют группу каталога, а новые подкаталоги — ещё и сам бит. Стандартный приём для общих каталогов команды.
- **umask** — маска, вычитаемая из запрошенных прав при создании. При типичном umask 022 файл, запрошенный с режимом 666, создаётся как 644, каталог с 777 — как 755. Маска наследуется процессами, поэтому демон, запущенный с umask 000, тихо создаёт файлы, доступные на запись всем.

2.5. Права суперпользователя не безграничны

root обходит проверки прав благодаря CAP_DAC_OVERRIDE, но на выполнение действует отдельное правило: **root может запустить файл, только если выставлен хотя бы один бит x** — любой из трёх. Файл с режимом 000 не запустит никто, включая root, пока права не изменят.

2.6. Запуск скрипта и запуск бинарника — разные требования

Это различие ловит почти всех, и именно оно проявилось в лабораторной работе.

Что запускаем	Нужен x	Нужен g	Почему
Бинарный файл ELF	да	нет	ядро отображает файл в память само, читать его процессу не нужно; режим 111 вполне рабочий
Скрипт с #!/bin/sh	да	да	ядро по строке #! запускает интерпретатор, а тот открывает файл обычным read и упирается в отсутствие права чтения
sh script.sh	нет	да	запускается интерпретатор, а скрипт для него — просто входные данные; бит x не проверяется вообще

Следствие, о котором нужно помнить

Режимы вида 100, 010, 001 и 111 на shell-скрипте бессмысленны: файл объявлен исполняемым, но запустить его нельзя. И обратное: проверка прав командой `sh` файл ничего не говорит о бите x, а `test -x` проверяет бит, но не фактическую исполнимость. Достоверный ответ даёт только настоящая попытка запуска файла напрямую.

2.7. Грабли, на которые наступают все

Симптом	Настоящая причина
«Я владелец, но не могу прочитать файл»	класс владельца применяется первым и исключительно; биты группы уже не смотрятся
«Добавил себя в группу, доступа нет»	список дополнительных групп фиксируется в учётных данных процесса при входе в систему; нужен новый вход, newgrp или sg
«Permission denied, хотя на файле 777»	нет бита x на одном из каталогов пути
«Скрипт с правами 755 не работает из-под другого пользователя»	у него нет x на каталоге или скрипт лежит на файловой системе, смонтированной с noexec
«Пользователь удалил файл, который ему был недоступен»	удаление определяется правами на каталог; нужен sticky bit
«Проверил доступ через access (), а open () упал»	access () использует реальный, а не эффективный UID, и создаёт гонку TOCTOU; правильный путь — просто пробовать open () и обрабатывать ошибку

3. Настоящие POSIX ACL: setfacl и getfacl

Трёх классов не хватает, как только нужно выдать права конкретному пользователю, не меняя владельца и не заводя новую группу. Для этого существуют расширенные POSIX ACL, поддерживаемые ext4, XFS и Btrfs. Именно они, а не биты `gwx`, в Linux называются списками контроля доступа.

Признак наличия ACL — символ `+` в конце строки вывода `ls -l`.

```
$ setfacl -m u:bob:rx,g:audit:r report.pdf
$ ls -l report.pdf
-rw-r-----+ 1 alice developers 40K Sep  6 12:00 report.pdf
$ getfacl report.pdf
# owner: alice
# group: developers
user::rw-          <- класс владельца
user:bob:r-x        <- именованный пользователь
group::r--          <- группа-владелец
group:audit:r--     <- именованная группа
mask::r-x           <- верхняя граница для всех записей выше, кроме владельца
other::---          <- остальные
```

3.1. Порядок разбора записей

1. Если субъект — владелец, применяется `user::` и проверка заканчивается.
2. Если есть запись `user:имя:` для этого субъекта, применяется она, ограниченная маской, и проверка заканчивается.
3. Иначе просматриваются `group::` и все `group:имя:`, куда входит субъект. Достаточно, чтобы **хоть одна** из них давала нужное право — здесь, в отличие от базовых битов, записи объединяются. Результат ограничен маской.
4. Иначе применяется `other::`.

Обратите внимание: в модели POSIX **нет записей запрета**. Список умеет только выдавать права. Это принципиальное отличие от NTFS и NFSv4, где запрет — полноправная запись.

3.2. Маска — главная ловушка POSIX ACL

Запись `mask::` задаёт верхнюю границу прав для именованных пользователей, именованных групп и группы-владельца. Владелец и «остальных» маска не касается. Коварство в том, что когда у файла есть ACL, **средняя триада в выводе `ls -l` показывает маску, а не права группы-владельца**.

```
$ getfacl --omit-header file | grep -E 'user:bob|mask'
user:bob:rw-
mask::rw-

$ chmod g-w file          # выглядит как «убрать запись у группы»
$ getfacl --omit-header file | grep -E 'user:bob|mask'
user:bob:rw-             # запись формально не изменилась
mask::r-x                 # но маска урезала её до r-x -> bob потерял запись
```

Правило безопасной работы

Никогда не правьте `chmod` файл, у которого есть ACL: одна команда тихо урежет права всем именованным записям сразу. Используйте `setfacl -m`, а маску пересчитывайте явно через `setfacl -R --recalc-mask` или ставьте нужное значение сами. Резервное копирование ACL: `getfacl -R dir > acl.txt`, восстановление: `setfacl --restore=acl.txt`.

3.3. Наследование: default ACL

Единственный механизм наследования прав в Linux — **default ACL** на каталоге. Он не влияет на доступ к самому каталогу, а служит шаблоном: новые файлы внутри получают его как свой действующий ACL, новые подкаталоги — и как действующий, и как собственный default.

```
# все новые файлы в /srv/shared будут доступны группе devs на запись
$ setfacl -d -m g:devs:rwx /srv/shared
$ setfacl -m g:devs:rwx /srv/shared      # плюс права на сам каталог

$ getfacl /srv/shared | grep default
default:group:devs:rwx
```

Учтите, что права, запрошенные при создании файла, всё равно пересекаются с default ACL: если программа создаёт файл с режимом 600, то default:group:devs:rwx даст группе не больше, чем rw-. Число записей ограничено: в ext4 весь ACL должен уместиться в один блок, что даёт около пятисот записей. Списки на сотни записей — признак того, что модель прав пора переносить на группы.

4. Другая ветвь: ACL в NFSv4 и NTFS

Windows и NFSv4 используют принципиально более выразительную модель. ACL — это упорядоченный список записей, каждая из которых либо разрешает, либо **запрещает** набор из десятков отдельных прав (не три, а, например, «удалить», «удалить подпапки», «сменить владельца», «изменить права», «дописать в конец»), и снабжена флагами наследования.

Свойство	POSIX + POSIX ACL	NTFS / NFSv4 ACL
Записи запрета	нет	есть, и обычно имеют приоритет
Накопление прав	нет для базовых классов, да для групповых записей ACL	да, права всех подходящих записей объединяются
Гранулярность	3 права	около 14 отдельных прав
Наследование	только default ACL на каталоге	флаги наследования на каждой записи, различают файлы и папки, глубину, «только для потомков»
Порядок записей	не важен, модель декларативная	важен; канонический порядок: явный запрет, явное разрешение, унаследованный запрет, унаследованное разрешение
Аудит	отдельный механизм (auditd)	встроен: SACL рядом с DACL

Практический вывод для разработчика: **не переносите ментальную модель одной системы на другую**. Код, который «выставляет права 0644» на смонтированном SMB-ресурсе или в контейнере с NFSv4, ведёт себя непредсказуемо, потому что трансляция между моделями приближительна. В Windows работают `icacls`, `Get-Acl` и `Set-Acl`; понятие владельца существует отдельно от DACL, и владелец всегда может вернуть себе права — как и в POSIX.

5. Что стоит рядом с ACL в Linux

5.1. Linux capabilities: конец монолитного root

Исторически проверка выглядела как «UID равен нулю или нет». Современное ядро разбивает всемогущество root примерно на четыре десятка независимых полномочий. Это позволяет дать процессу ровно одно нужное право вместо полного набора.

Полномочие	Что даёт	Типичное применение
CAP_NET_BIND_SERVICE	занимать порты ниже 1024	веб-сервер на 80/443 без запуска от root
CAP_NET_RAW	сырые сокеты	ping, снятие трафика
CAP_DAC_OVERRIDE	обходить проверку прав файлов	системы резервного копирования
CAP_CHOWN	менять владельца	распаковщики образов
CAP_SYS_ADMIN	огромный разнородный набор операций	практически эквивалент root; выдавать его — значит не ограничивать ничего

```
# вместо setuid root: выдать бинарнику одно конкретное полномочие
$ sudo setcap cap_net_bind_service=+ep /usr/local/bin/myserver
$ getcap /usr/local/bin/myserver
/usr/local/bin/myserver cap_net_bind_service=ep

# что есть у процесса
$ grep Cap /proc/self/status

# в контейнерах Docker по умолчанию оставляет узкий набор
$ docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myimage
```

Правило для контейнеров

--privileged и CAP_SYS_ADMIN — это отказ от изоляции, а не её настройка. В Kubernetes тот же смысл имеют securityContext.privileged: true и allowPrivilegeEscalation: true. Начинайте с --cap-drop ALL и добавляйте по одному, когда что-то действительно не работает.

5.2. Мандатный слой: SELinux и AppArmor

Это модули безопасности ядра (LSM), которые накладываются **поверх** прав POSIX. Порядок такой: сначала ядро проверяет DAC, и если DAC отказал — всё, отказ окончателен. Если DAC разрешил, вопрос задаётся модулю. **MAC умеет только запрещать то, что разрешил DAC, но никогда не разрешает то, что DAC запретил.**

- **SELinux** маркирует субъекты и объекты метками вида system_u:object_r:httpd_sys_content_t:s0 и разрешает взаимодействие только описанным в политике парам типов. Метки видны через `ls -Z` и `ps -Z`, режим переключается `getenforce / setenforce`, а причину отказа ищут в журнале аудита через `ausearch -m avc` и `audit2allow`.
- **AppArmor** описывает профили по путям файлов, а не по меткам: проще в чтении и слабее по гарантиям. Состояние — `aa-status`, профили лежат в `/etc/apparmor.d`.

Классический сценарий, который стоит узнавать сразу: приложение с правильными правами POSIX всё равно получает отказ, в логах приложения невнятная ошибка доступа, а настоящая причина видна только в журнале аудита ядра. Первым делом проверяйте, не работает ли MAC в режиме принуждения.

5.3. seccomp: ограничение по системным вызовам

Отдельный от прав механизм, который стоит знать в одном ряду: фильтр разрешённых системных вызовов процесса. Права отвечают на вопрос «к какому объекту можно обратиться», seccomp — «какие операции ядра вообще доступны». Профиль Docker по умолчанию блокирует несколько десятков вызовов, и это одна из причин, почему часть системных утилит не работает в контейнере, несмотря на root внутри него.

6. Разграничение доступа в СУБД

СУБД — это отдельная система контроля доступа со своими субъектами и объектами, никак не связанная с правами операционной системы. Разберём на примере PostgreSQL, где модель наиболее полная; в MySQL и Oracle принципы те же, отличается синтаксис и гранулярность.

6.1. Роль — это и пользователь, и группа

В PostgreSQL нет отдельных сущностей «пользователь» и «группа»: есть роль. Роль с атрибутом LOGIN может подключаться, роль без него служит контейнером прав. Членство задаётся командой GRANT роль TO роль.

```
CREATE ROLE app_reader NOLOGIN;  
GRANT USAGE ON SCHEMA app TO app_reader;  
GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_reader;  
  
CREATE ROLE report_svc LOGIN PASSWORD '...';  
GRANT app_reader TO report_svc;
```

Привилегии наследуются, атрибуты — нет

GRANT роль TO пользователю передаёт привилегии (SELECT, INSERT и прочие), но не передаёт атрибуты роли: SUPERUSER, CREATEDB, CREATEROLE, BYPASSRLS, LOGIN. Их назначают учётной записи напрямую через ALTER ROLE. Об это спотыкаются регулярно: роль выдали, а администратор всё равно не видит всех строк, потому что BYPASSRLS остался у роли-контейнера.

6.2. Уровни, на которых выдаются права

Уровень	Пример	Зачем
Кластер	GRANT pg_read_all_data TO ...	предопределённые роли для служебных задач
База данных	GRANT CONNECT ON DATABASE app_db TO ...	кто вообще может подключиться
Схема	GRANT USAGE, CREATE ON SCHEMA app TO ...	разделение подсистем внутри одной базы
Таблица	GRANT SELECT, INSERT ON app.orders TO ...	основной рабочий уровень
Столбец	GRANT SELECT (id, title) ON app.orders TO ...	скрыть отдельные поля, не разделяя таблицу
Строка	политика CREATE POLICY ... USING (...)	многоарендность, доступ к своим записям

6.3. PUBLIC и привилегии по умолчанию

PUBLIC — псевдороль, в которую входят все. Часть прав достаётся ей автоматически, и без явного отзыва разграничение оказывается декоративным.

- CONNECT на новую базу данных выдан PUBLIC. Пока вы его не отозвали, подключиться может любая учётная запись кластера — в том числе к служебной базе postgres, откуда видны каталоги всего кластера.
- EXECUTE на новые функции выдан PUBLIC.

- До PostgreSQL 15 право CREATE на схему public было у PUBLIC, то есть любой пользователь мог создавать там объекты. В 15-й версии это наконец убрали, но на старых базах наследие остаётся.

```
REVOKE ALL ON DATABASE app_db FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE CONNECT ON DATABASE postgres FROM PUBLIC;

-- права на будущие таблицы, а не только на существующие
ALTER DEFAULT PRIVILEGES IN SCHEMA app
    GRANT SELECT ON TABLES TO app_reader;
```

Про ALTER DEFAULT PRIVILEGES забывают чаще всего: GRANT ... ON ALL TABLES действует на таблицы, существующие в момент выполнения. Таблица, созданная завтра миграцией, прав не получит, и приложение сломается после деплоя, а не в момент настройки.

6.4. Построчная защита

RLS переносит проверку доступа в саму базу: даже если приложение забудет добавить условие в запрос, лишние строки не вернутся. Это лучший известный способ защититься от утечки данных между арендаторами.

```
ALTER TABLE app.orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON app.orders
    FOR ALL TO app_user
    USING      (tenant_id = current_setting('app.tenant')::uuid)
    WITH CHECK (tenant_id = current_setting('app.tenant')::uuid);

-- владелец таблицы политику обходит, если не потребовать обратное
ALTER TABLE app.orders FORCE ROW LEVEL SECURITY;
```

- USING фильтрует то, что читается и изменяется; WITH CHECK проверяет то, что записывается. Без второго пользователь сможет вставить строку чужого арендатора, хотя и не увидит её.
- Владелец таблицы и роли с BYPASSRLS политику игнорируют. Если приложение ходит в базу под владельцем схемы, RLS не работает вовсе — нужен FORCE ROW LEVEL SECURITY или отдельная непривилегированная роль.
- Значение арендатора передаётся через SET LOCAL в рамках транзакции. Обычный SET в пуле соединений утечёт в следующий запрос другого пользователя — это реальный источник инцидентов.

6.5. Грабли PostgreSQL

Ситуация	Что происходит на самом деле
GRANT от лица не-владельца «прошёл»	PostgreSQL выдаёт предупреждение, а не ошибку, и код возврата клиента остаётся нулевым; проверять нужно фактический результат, а не успешность команды
Роль не удаляется	DROP ROLE падает, пока за ролью числятся объекты или выданные ей права; нужны REASSIGN OWNED и DROP OWNED
Приложение внезапно получило больше прав	проверьте цепочку членств: привилегии наследуются транзитивно, \du показывает список ролей
«Права выданы, а доступа нет»	почти всегда не хватает USAGE на схему: право на таблицу без права на схему бесполезно

```
\dp app.*          -- права на таблицы схемы
\du                -- роли и членства
-- расшифровка aclitem: получатель=права/кто_выдал
-- user_sub=arwd/postgres -> SELECT, INSERT, UPDATE, DELETE
SELECT * FROM information_schema.role_table_grants WHERE grantee = 'app_user';
```

7. Авторизация в приложении

Права файловой системы и СУБД защищают инфраструктуру. Но основной объём решений о доступе принимает само приложение: может ли этот пользователь открыть этот заказ, этот документ, этот проект. Здесь и находится источник большинства реальных утечек.

7.1. Три поколения моделей

	RBAC	ABAC	ReBAC
Вопрос	какая у пользователя роль?	какие атрибуты у запроса?	как пользователь связан с объектом?
Правило	admin может всё в проекте	разрешить, если <code>user.dept == doc.dept</code> и время рабочее	разрешить, если пользователь — editor документа или owner папки, в которой он лежит
Сильная сторона	просто, понятно, легко объяснить аудитору	гибко, учитывает контекст	естественно описывает иерархии и совместный доступ
Слабая сторона	взрыв числа ролей при росте гранулярности	трудно ответить, кто имеет доступ к объекту	нужен отдельный сервис и продуманная модель данных

Практический маршрут развития почти всегда одинаков: начинается с RBAC, затем к ролям добавляются условия (получается ABAC), и при появлении совместного доступа с наследованием («папка → документ», «организация → проект → задача») система приходит к ReBAC.

7.2. Zanzibar: как это устроено в больших системах

Zanzibar — система авторизации Google, описанная в статье 2019 года и обслуживающая Drive, YouTube, Calendar и остальные продукты. Её стоит знать, потому что она задала стандарт де-факто, и все современные открытые решения повторяют её модель.

- Права хранятся как **кортежи отношений** вида объект#отношение@субъект, например `document:readme#editor@user:alice`. Это ACL, вынесенный в отдельную базу и превращённый в граф.
- Отношения **вычисляются рекурсивно**: «читатель документа» может быть определён как «все, кто явно читатель, плюс все редакторы, плюс все читатели родительской папки». Наследование задаётся схемой, а не копированием прав.
- Проверка — это обход графа с ответом «да/нет» за единицы миллисекунд, плюс отдельные операции «перечислить объекты, доступные субъекту» и «перечислить субъектов, имеющих доступ к объекту».
- Согласованность решается токенами версии (в статье — *zookie*): чтобы после отзыва прав никто не получил доступ по устаревшему кэшу.

Открытые реализации, которые встречаются в проектах: **SpiceDB**, **OpenFGA** (в CNCF, вырос из Auth0) и **Ory Keto**. Для политик как кода в другом стиле применяются **OPA** с языком Rego (популярен в Kubernetes и в проверках инфраструктуры) и **Cedar** от AWS (лежит в основе Amazon Verified Permissions).

7.3. Самая частая уязвимость: BOLA

Broken Object Level Authorization, она же IDOR, занимает первое место в OWASP API Security Top 10. Суть проста: приложение проверяет, что пользователь аутентифицирован, но не проверяет, что запрошенный объект принадлежит именно ему.

```
# уязвимо: идентификатор пришёл от клиента и используется как есть
@app.get("/api/orders/{order_id}")
def get_order(order_id: int, user = Depends(current_user)):
    return db.query(Order).get(order_id)

# правильно: объект ищется в области видимости пользователя
@app.get("/api/orders/{order_id}")
def get_order(order_id: int, user = Depends(current_user)):
    order = db.query(Order).filter_by(id=order_id, tenant_id=user.tenant_id).one_or_none()
    if order is None:
        raise HTTPException(404)      # 404, а не 403: не подтверждаем существование
    return order
```

- Проверяйте доступ **на каждом запросе и по идентификатору объекта**, а не один раз при входе и не в интерфейсе. Скрытая кнопка не является средством разграничения доступа.
- Ограничивайте выборку в самом запросе к базе, а не фильтруйте результат в коде: забытый фильтр в одном месте — это утечка.
- Держите RLS или аналог как второй рубеж на случай, если фильтр всё-таки забыли.
- Отвечайте 404 там, где 403 раскрыл бы факт существования чужого объекта.
- Непредсказуемые идентификаторы (UUID вместо последовательных чисел) — полезная мера, но это затруднение перебора, а не разграничение доступа.

8. AWS: политики, ACL и порядок их вычисления

AWS — самая сложная из распространённых систем доступа, потому что решение складывается из шести независимых источников. Понимать нужно не синтаксис (он простой), а именно порядок вычисления.

8.1. Источники решения

Тип политики	К чему прикреплена	Пример
Identity-based	к пользователю, группе или роли	разрешить роли приложения s3:GetObject на конкретный бакет
Resource-based	к самому ресурсу	bucket policy у S3, key policy у KMS, trust policy у роли IAM
Service Control Policy	к организации, подразделению, аккаунту	запретить всему аккаунту любые действия вне региона eu-central-1
Permission boundary	к пользователю или роли	верхняя граница прав, которую не превысит даже администратор этого аккаунта
Session policy	к временным учётным данным	сузить права при AssumeRole на время сессии
ACL	к объекту S3 или подсети VPC	устаревший механизм в S3; сетевые списки в VPC

8.2. Порядок вычисления — то, что нужно помнить наизусть

1. По умолчанию — **запрет**. Отсутствие разрешения равносильно отказу.
2. **Явный Deny в любой из политик выигрывает всегда** и не может быть перекрыт никаким Allow.
3. Проверяются SCP организации: если они не разрешают действие, всё останавливается здесь. SCP не выдают права, они лишь ограничивают.
4. Проверяются граница прав и политика сессии, если они заданы.
5. Проверяются identity- и resource-политики. **В пределах одного аккаунта** достаточно разрешения в любой из них. **Для доступа между аккаунтами** разрешение нужно в обоих: в политике вызывающей стороны и в политике ресурса.

Ловушка с условиями в запретах

Правило «запретить всё без MFA» с условием "Bool": {"aws:MultiFactorAuthPresent": "false"} не сработает для вызовов, где этот ключ вообще отсутствует: условие не совпадёт, и запрет не применится. Нужен BoolIfExists. Это классический источник дыр в политиках, которые «выглядят правильно».

8.3. S3: почему ACL объявлены устаревшими

В S3 исторически было два механизма: ACL на бакете и объекте (наследие первых лет сервиса, до появления IAM) и bucket policy. Они действовали параллельно, и разобраться, откуда взялся публичный доступ, было тяжело. Практически все громкие утечки данных из S3 связаны именно с этим.

- AWS рекомендует **полностью отключать ACL** через настройку Object Ownership в режиме Bucket owner enforced. Тогда права определяются только политиками.

- С апреля 2023 года у новых бакетов ACL отключены, а Block Public Access включён по умолчанию.
- Block Public Access — отдельный рубеж поверх политик: он блокирует публичный доступ независимо от того, что написано в политике. Включать на уровне аккаунта.
- Предподписанные ссылки — это s3ability: они действуют от имени того, кто их выпустил, живут до истечения срока и не отзываются. Срок ставьте минутами, а не сутками.

8.4. Security Group против Network ACL

Различие спрашивают на любом собеседовании по AWS, и путаница в нём приводит к реальным сбоям.

	Security Group	Network ACL
Уровень	сетевой интерфейс (инстанс)	подсеть
Состояние	stateful : ответный трафик разрешается автоматически	stateless : нужно явно разрешать обе стороны, включая эфемерные порты 32768–65535
Правила	только разрешающие	разрешающие и запрещающие
Обработка	все правила, объединение результатов	по номеру, сверху вниз, срабатывает первое подходящее
По умолчанию	весь входящий запрещён, весь исходящий разрешён	стандартный список разрешает всё, созданный вручную — запрещает всё

Самая частая ошибка

Настроили Network ACL «разрешить входящий 443» и удивляетесь, что ничего не работает. Список без состояния: ответы сервера уходят с порта 443 на эфемерный порт клиента, и для них нужно отдельное исходящее правило. По умолчанию используйте Security Group, а Network ACL — только для грубых запретов на уровне подсети.

8.5. Пути повышения привилегий и типовые ошибки

Ошибка	Чем опасна	Как правильно
"Action": "*", "Resource": "**"	роль приложения становится администратором	перечислять действия явно, ограничивать ARN ресурсов
iam:PassRole без ограничений	позволяет запустить сервис с любой ролью аккаунта и получить её права	ограничивать конкретными ARN ролей и условием iam:PassedToService
Долгоживущие ключи доступа	утекают в репозитории и образы, не имеют срока годности	роли IAM, а для CI — федерация через OIDC без ключей вообще
Доверие стороннему аккаунту без ExternalId	проблема «запутавшегося заместителя»: чужой клиент того же подрядчика получает доступ к вашему аккаунту	sts:ExternalId в trust policy, а для сервисов — условия aws:SourceArn и aws:SourceAccount
Доступ к метаданным инстанса из приложения	любая SSRF-уязвимость превращается в кражу учётных данных роли	требовать IMDSv2 и выставлять hop limit равным единице

```
# что реально может принципал
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111122223333:role/app \
  --action-names s3:GetObject --resource-arns arn:aws:s3:::my-bucket/*

# какими сервисами роль пользовалась на самом деле -> чем урезать политику
aws iam get-service-last-accessed-details --job-id ...

# найти ресурсы, открытые наружу
aws accessanalyzer list-findings --analyzer-arn ...
```

9. GCP: иерархия и объединение политик

GCP устроен иначе, и главное отличие сформулировать нужно чётко: **политики наследуются вниз по иерархии ресурсов и объединяются, а дочерний уровень не может отобрать то, что выдал родительский**. Это прямо противоположно поведению SCP в AWS.

9.1. Иерархия

```
Организация
├── Папка (Folder)
│   └── Проект (Project)
│       └── Ресурс (бакет, ВМ, набор данных)
```

Действующие права = объединение привязок на всех уровнях цепочки.

Политика (IAM policy) — это набор привязок «роль ↔ участники». Участником может быть пользователь, группа Google, домен, сервисная учётная запись или пул федеративных удостоверений. Выдали роль Viewer на уровне организации — она действует во всех проектах, и убрать её внутри проекта настройками проекта невозможно.

9.2. Чем ограничивать, если объединение нельзя отменить

- **IAM Deny Policies** — отдельный механизм явных запретов, который вычисляется **до** разрешений и перекрывает их. Появился именно потому, что чистого объединения не хватало.
- **Organization Policy Constraints** — ограничения на уровне конфигурации, а не на уровне прав: например, `constraints/storage.publicAccessPrevention` запрещает делать бакеты публичными вообще, `constraints/iam.allowedPolicyMemberDomains` разрешает выдавать права только участникам своих доменов, `constraints/compute.vmExternalIpAddress` запрещает внешние адреса.
- **IAM Conditions** — условия на языке CEL, привязанные к привязке роли: по времени, по имени ресурса, по тегам. Это элемент ABAC внутри ролевой модели.

9.3. Роли и разрешения

Вид роли	Пример	Комментарий
Базовая (basic)	<code>roles/owner</code> , <code>roles/editor</code> , <code>roles/viewer</code>	наследие ранних лет, крайне широкие; в продакшене не применять
Предопределённая	<code>roles/storage.objectViewer</code>	рабочий вариант по умолчанию, сопровождается самим Google
Пользовательская	набор из <code>storage.objects.get</code> , <code>.list</code>	когда предопределённая даёт лишнее; требует сопровождения при появлении новых разрешений

9.4. Сервисные учётные записи — главный вектор эскалации

Сервисная учётная запись в GCP одновременно является и субъектом, и ресурсом, на который есть свои права. Отсюда два опасных разрешения.

- `iam.serviceAccounts.actAs` — позволяет запустить ресурс от имени учётной записи и тем самым получить её права. Прямой аналог `iam:PassRole` в AWS.

- `roles/iam.serviceAccountTokenCreator` — позволяет выпустить токен от имени чужой учётной записи, то есть выдать себя за неё.
- Ключи сервисных учётных записей в виде JSON-файлов — источник утечек. Замена: Workload Identity Federation для внешних систем и Workload Identity для нагрузок в GKE, то есть работа вообще без файлов ключей.

9.5. Cloud Storage: uniform против fine-grained

В GCS, как и в S3, есть наследие в виде ACL на объектах. Рекомендуемый режим — **uniform bucket-level access**: он полностью отключает ACL объектов, и доступ определяется только политиками IAM. Режим `fine-grained` оставляет старые ACL и создаёт ту же проблему двух параллельных источников прав, что и в S3. Signed URL здесь играет ту же роль `sarability`, что и предподписанная ссылка в AWS.

```
# кто и что может в проекте
gcloud projects get-iam-policy my-project --format=json

# почему конкретный запрос был разрешён или отклонён
gcloud policy-troubleshoot iam //storage.googleapis.com/projects/_/buckets/b \
  --principal-email=svc@my-project.iam.gserviceaccount.com \
  --permission=storage.objects.get
```

9.6. Сводное сравнение подходов

	AWS	GCP
Базовое правило	запрет по умолчанию, разрешения складываются	запрет по умолчанию, разрешения складываются
Наследование	нет иерархии политик; ограничение сверху — SCP	иерархия организация → папка → проект, объединение вниз
Явный запрет	встроен в любую политику, всегда побеждает	отдельный механизм Deny Policies
Ограничение сверху	SCP и <code>permission boundary</code>	Organization Policy и Deny Policies
Атрибуты и условия	блок <code>Condition</code> в политике	IAM Conditions на языке CEL
Наследие ACL	S3 object ACL, рекомендуется отключать	GCS object ACL, рекомендуется <code>uniform access</code>

10. Kubernetes: RBAC и сетевые политики

10.1. RBAC умеет только разрешать

В Kubernetes нет запрещающих правил вообще: права складываются, и если хоть одна привязка разрешает действие, оно разрешено. Убрать право можно только убрав привязку.

Объект	Область	Назначение
Role	пространство имён	набор правил «глаголы x ресурсы»
ClusterRole	весь кластер	то же, плюс ресурсы без пространства имён: узлы, тома, сами роли
RoleBinding	пространство имён	связывает субъекта с Role или с ClusterRole, но действие ограничивается пространством имён
ClusterRoleBinding	весь кластер	связывает субъекта с ClusterRole во всём кластере

```
# всегда начинайте отладку прав с этого
kubectl auth can-i --list --as=system:serviceaccount:prod:api
kubectl auth can-i delete pods --as=system:serviceaccount:prod:api -n prod
```

10.2. Пути повышения привилегий, которые нужно знать

- Право создавать поды в пространстве имён равносильно праву запускать код от имени любой сервисной учётной записи этого пространства: достаточно смонтировать её токен в под.
- Глаголы `escalate` и `bind` позволяют выдать себе права выше собственных, обходя обычную защиту от эскалации.
- `pods/exec` и `pods/portforward` — это фактический доступ внутрь чужой нагрузки, а не «только чтение».
- Доступ к секретам в пространстве имён — это доступ ко всему, что этими секретами защищено. Выдача `get secrets` в масштабе кластера равносильна выдаче администратора.
- Монтирование `hostPath` с корнем узла или `privileged: true` — выход из контейнера на узел.

10.3. NetworkPolicy: семантика по умолчанию

Здесь тоже нет запретов, но логика инвертирована и потому неочевидна: **пока под не выбран ни одной политикой, разрешено всё. Как только его выбрала хотя бы одна политика для данного направления, всё остальное в этом направлении запрещено.** Поэтому типовая практика — сначала применить политику «запретить весь входящий трафик» ко всем подам пространства имён, а затем точно разрешать.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}           # выбирает все поды пространства имён
  policyTypes: [Ingress]    # входящий трафик теперь запрещён по умолчанию
```

Проверьте, что используемый сетевой плагин вообще поддерживает NetworkPolicy: у части CNI объекты создаются, но не действуют, и политика существует только на бумаге.

11. Сквозные принципы

1. **Запрет по умолчанию.** Система должна начинать с нуля прав и открывать доступ явно. Любая схема, где по умолчанию разрешено, деградирует за месяцы.
2. **Минимальные привилегии.** Права выдаются под задачу, а не «чтобы точно работало». Практический приём: выдать минимум, собрать реальные обращения из журналов и расширить по факту.
3. **Отказ закрывает.** Если сервис авторизации недоступен, запрос должен отклоняться, а не проходить. Обратное поведение превращает сбой в полное отключение защиты.
4. **Короткоживущие учётные данные.** Токен на пятнадцать минут вместо ключа без срока годности; федерация вместо файла с ключом в CI. Это снимает саму проблему отзыва прав у sarability.
5. **Разделение обязанностей.** Тот, кто выдаёт права, и тот, кто ими пользуется, — разные роли. Отдельная учётная запись «на случай аварии» с журналируемым доступом.
6. **Один источник истины.** Два параллельных механизма прав на одном ресурсе (ACL плюс политика) — гарантированная путаница; именно поэтому и AWS, и Google рекомендуют отключать наследие ACL в своих хранилищах.
7. **Проверка на сервере, на каждом запросе, по идентификатору объекта.** Клиентский интерфейс не является средством разграничения доступа.
8. **Наблюдаемость.** Нужно уметь ответить на два вопроса: «кто имеет доступ к этому объекту» и «к чему имеет доступ этот субъект». Если модель не позволяет ответить, ею невозможно управлять.
9. **Аудит и пересмотр.** Права накапливаются: человек сменил команду, а старые доступы остались. Регулярный пересмотр — часть эксплуатации, а не разовое мероприятие.

Что стоит помнить как главный вывод

Права — это не набор битов, а слоёная система решений: ядро, мандатный модуль, СУБД, приложение, облачная политика. Отказ на любом слое означает отказ целиком, поэтому отладка всегда начинается не с вопроса «какие права выдать», а с вопроса «какой именно слой отказал и почему».

12. Шпаргалка команд

Linux: базовые права

```
ls -l /path # владелец, группа, биты; + означает наличие ACL
ls -ld /path # то же для самого каталога, а не его содержимого
stat -c '%A %a %U %G %n' file # права в символьном и восьмеричном виде
namei -l /a/b/c/file # права на каждом компоненте пути: где именно отказ
id user; groups user # членство в группах
find / -perm -4000 -type f # все setuid-бинарники в системе
chmod 750 dir; chmod g+s dir # права и наследование группы
chown user:group file
umask # маска текущей оболочки
```

Linux: расширенные ACL и полномочия

```
getfacl file # полный список записей
setfacl -m u:bob:rx file # выдать именованному пользователю
setfacl -x u:bob file # убрать запись
setfacl -d -m g:devs:rwX dir # шаблон для новых объектов внутри каталога
setfacl -b file # снять расширенный ACL целиком
getfacl -R dir > acl.txt # резервная копия
setfacl --restore=acl.txt # восстановление

getcap /path/to/bin # полномочия файла
setcap cap_net_bind_service=+ep /path/to/bin
grep Cap /proc/PID/status # полномочия процесса

getenforce; ausearch -m avc -ts recent # SELinux: режим и отказы
aa-status # AppArmor: активные профили
```

PostgreSQL

```
\du # роли и членства
\dp schema.* # привилегии на таблицы
\dtn+ # привилегии на схемы
SELECT * FROM information_schema.role_table_grants WHERE grantee = 'app_user';
SELECT rolname, rolsuper, rolcreatorole, rolbypassrls FROM pg_roles;
SELECT * FROM pg_policies WHERE tablename = 'orders'; -- политики RLS
```

Облака и Kubernetes

```
# AWS
aws sts get-caller-identity
aws iam simulate-principal-policy --policy-source-arn ARN --action-names ACTION
aws s3api get-bucket-policy-status --bucket B # публичен ли бакет
aws s3api get-bucket-ownership-controls --bucket B # отключены ли ACL

# GCP
gcloud projects get-iam-policy PROJECT
gcloud policy-troubleshoot iam RESOURCE --principal-email=... --permission=...
gcloud storage buckets describe gs://B --format='value(uniform_bucket_level_access)'

# Kubernetes
kubectl auth can-i --list --as=system:serviceaccount:NS:SA
kubectl get clusterrolebinding -o wide | grep cluster-admin
```

13. Что почитать дальше

- Saltzer J., Schroeder M. The Protection of Information in Computer Systems, 1975 — статья, из которой происходят принципы минимальных привилегий, запрета по умолчанию и разделения обязанностей.
- Lampson B. Protection, 1971 — исходная работа о матрице доступа.
- Pang R. и др. Zanzibar: Google's Consistent, Global Authorization System, USENIX ATC 2019 — модель, на которой построены SpiceDB и OpenFGA.
- `man 7 capabilities`, `man 5 acl`, `man 1 setfacl` — точная и исчерпывающая справка по механизмам Linux.
- AWS IAM Policy Evaluation Logic — официальное описание порядка вычисления политик; стоит прочитать целиком хотя бы раз.
- Документация Google Cloud IAM, разделы об иерархии ресурсов и о Deny Policies.
- OWASP API Security Top 10 — практический перечень того, как авторизация ломается в реальных приложениях.